

=====

BD1 - ENUNCIADOS DE PARCIALES - MySQL - (2022-2024)

=====

1) Cursada 2023 - 1ra fecha - 15/11

Esquema:

USUARIO (dni, email, nombre)
ACTIVIDAD (#actividad, #proyecto, nombre, detalle_actividad, condiciones_actividad)
PROYECTO (#proyecto, nombre, presupuesto, cantidad_días)
COMENTARIO (#comentario, dni, #actividad, valoración, detalle_comentario)

1. Se modificó el esquema anterior, presentado en el ejercicio de Álgebra Relacional, para agregar a la tabla ACTIVIDAD, una nueva columna llamada "cantidad_comentarios", resultando:

ACTIVIDAD (#actividad, #proyecto, nombre, detalle_actividad, condiciones_actividad, cantidad_comentarios)

Escriba un Trigger para que con cada nuevo comentario registrado se actualice la nueva columna de la actividad (cantidad_comentarios).

2. Enuncie y explique dos motivos por los cuales necesitaría definir una vista.

=====

2) Cursada 2023 - 2da fecha - 29/11

Esquema:

EMPLEADO (#empleado, email, nombre, años_antigüedad)
PREMIO (#premio, descripción_premio)
RECIBE_PREMIO (#premio, #empleado, fecha_premio)
SANCION (#sancion, descripción_sancion)
RECIBE_SANCION (#sancion, #empleado, fecha_sancion, gravedad)

Se modificó el esquema anterior, presentado en el ejercicio de Álgebra Relacional, para agregar la siguiente tabla:

AUDITORÍA (#auditoria, #empleado, #sanción, prioridad)

Escriba un trigger para que, con cada nueva sanción registrada a un empleado, si esta es de gravedad "Alta" se agregue una nueva tupla a la tabla AUDITORIA donde se registre la correspondiente información con la prioridad "Inmediata".

=====

3) Cursada 2023 - 3ra fecha - 13/12

Esquema:

DUEÑO (id_dueño, nombre, teléfono, dirección, dni)
CHOFER (id_chofer, nombre, teléfono, dirección, fecha_licencia_desde, fecha_licencia_hasta, dni)
AUTO (patente, id_dueño, id_chofer, marca, modelo, año)

Utilizando el esquema presentado en el ejercicio de Álgebra Relacional, escriba un Store Procedure en MySQL llamado "nuevo_chofer". Este debe declarar como parámetro un dni y dos fechas (licencia_desde y licencia_hasta), y debe generar, dentro de una transacción, un nuevo chofer con los datos recuperados del dueño con dni igual al recibido como parámetro. Asuma que este dni ya existe en DUEÑO (esquema del ej.de AR).

=====

4) Cursada 2024 - 1ra fecha - 13/11

Esquema:

Cliente (#cliente, dni, nombre, dirección, teléfono)
Perito (#perito, nombre_perito, apellido_perito)
Siniestro (#siniestro, fecha_denuncia, descripción_siniestro, #taller, #póliza)
Póliza (#póliza, fecha_inicio, fecha_fin, #cliente)
Peritaje (fecha_peritaje, #perito, #siniestro, evaluación)
Tipo_Seguro (#tipo_seguro, nombre_tipo_seguro)
Póliza_Tipo_Seguro (#póliza, #tipo_seguro)

Se ha creado una base de datos siguiendo el esquema definido en el ejercicio de Álgebra Relacional, con una relación uno a uno para las tablas y atributos. En esta base de datos, crear un stored procedure que sirva para agregar, de forma transaccional, un nuevo siniestro, junto al registro del perito, que realiza el peritaje. El stored procedure debe recibir los siguientes parámetros: #póliza, descripción_siniestro, #taller, #perito, nombre_perito, apellido_perito, evaluación. El #siniestro es un autoincremental. La fecha_denuncia será la fecha actual.

5) Cursada 2024 - 2da fecha - 27/11

Se dispone de una base de datos para la gestión de subastas judiciales, permitiendo registrar la participación de oferentes, previamente inscritos, en estas. Para cada subasta se poseen diferentes ofertas, cada una con un monto superior a la oferta previa, de estas se registra, además del monto, el oferente correspondiente y la fecha. En la tabla "Oferta" siempre se guarda la última oferta válida. Es necesario registrar un historial de cada oferta realizada.

Subasta_Judicial (#subasta, fecha_comienzo, fecha_fin)
Inscripto_Subasta (#inscripto_subasta, #oferente, #subasta, fecha_inscripción)
Oferente (#oferente, nombre, apellido, cull, localidad)
Oferta (#oferta, #subasta, monto, fecha_oferta) -> asumir que existe #oferente en Oferta
Oferta_Historial (#oferta_historial, #oferente, #subasta, monto, fecha_oferta, fecha_auditoria)

Generar un trigger que guarda la última oferta en la tabla "Oferta_Historial" antes de grabar la nueva oferta superadora.

6) Sin Fecha 1

Se dispone de una base de datos que registra los envíos a usuarios por parte de una empresa de correos. De cada envío, se sabe el usuario que lo solicita además de la información concreta del envío. Además, la empresa dispone de un sistema de puntos para los usuarios, que luego permite otorgar beneficios. La estructura de la base de datos es la siguiente:

Usuario (#usuario, nombre, email, teléfono, puntaje)
Envío (#envio, #usuario, direccion_entrega, fecha_solicitud, información_destinatario, estado_envio)
Beneficio (#beneficio, descripción, puntos_requeridos)
Canje (#canje, #usuario, #beneficio, fecha_canje)

Generar un trigger que aumente en 5 puntos el puntaje del usuario luego de que este registre un nuevo envío.

7) Sin Fecha 2

1. Considerando las propiedades ACID de las transacciones y priorizando que esto se cumpla, ¿es necesario usar transacciones cuando se ejecuta una sentencia UPDATE? Justifique su respuesta.

2. Explique 2 motivos por los cuales necesitaría definir una vista.

3. En una base de datos con las siguientes tablas

CONTENIDO(#id_contenido, título, sinopsis)

VISUALIZACION(#id, timestamp, #id_contenido)

Hace falta crear un usuario para el frontend de la aplicación que sea capaz de obtener información sobre los contenidos, y registrar visualizaciones (pero no consultarlas). ¿Qué permisos le asignaría a cada tabla, para maximizar la seguridad de la BD?
